Making KKT practical

Implementation archaeology, measured scaling, and graph structure
Francisco Thiesen's Karger–Klein–Tarjan repository · 10–11 October 2026

This report asks three distinct questions: whether the original implementation faithfully realized KKT's local complexity bounds; whether the current implementation *measures* like linear time; and which graph structures make it competitive in practice. The answers come from source inspection, certified benchmark outputs, separate work counters, two complete new timing campaigns, and a controlled comparison with repeated compacting Borůvka steps.

The main document is this Markdown file. The HTML edition adds a fully offline, interactive 3D graph explorer. The standalone explorer exposes actual contraction and filtering results for seven complete small graphs. A paginated companion checks the requested 40-page limit.

Measured source. New benchmark infrastructure is revision `c9ed4ca`. The production KKT algorithm is unchanged from `2d4fbf1`; all four algorithm implementations, the earlier fixtures, and the original timing boundary were checked against `1a3fc3c`. Raw evidence, precise commands, binary/source hashes and validation records are retained in the new experiment directory.

The answer in three parts. The old implementation had two genuine superlinear bookkeeping/preprocessing defects, plus a recursive RNG-reset problem. Repairing them and reducing allocation/hash costs made KKT useful on several structured inputs. But the current wall-clock curves do **not** measure as uniformly linear, and larger random graphs still do not show general-purpose dominance.

In the new study, KKT leads Kruskal, Prim and the repository's ordinary Borůvka on **21 of 52 cases in both campaigns, on all three graph medians**. The strongest new family has a small bipartite side: at 262,144 vertices, k=16 takes 0.463 / 0.475 times Kruskal's time. Leaf-rich cores produce smaller gains; all tested meshes lose to Kruskal.

The decisive qualification is the continuation experiment: simply repeating KKT's own compacting Borůvka step beats KKT in **18 of 20 cases in both runs**. The only repeated KKT median lead is about 1% and does not hold across every graph. These experiments establish practical wins for the current implementation against the original baselines; they do not establish that randomized sampling is the source of those wins.

These are results for the repository's implementations on one Apple M3 Max, using the stated build profiles and fixtures. The report retains losses, historical limitations, and the distinction between an algorithmic explanation and a measured causal effect.

1. Which problem are we comparing?

KKT, Kruskal, Prim and Borůvka solve the minimum spanning tree problem: connect every vertex while minimizing the **sum of selected edge weights**. The user's reference to Dijkstra was clarified to mean Borůvka. Dijkstra instead minimizes the distance from one source to each other vertex.

A three-vertex example makes the distinction concrete. Let `AB=2`, `BC=3`, `AC=4`, with source A. Dijkstra's shortest-path tree contains AB and AC, total weight 6. The MST contains AB and BC, total weight 5, but its A-to-C route costs 5 rather than the shortest possible 4. Ranking those outputs as competing MST solutions would compare different objectives.

Throughout this report:

Symbol	Meaning
`n`, `m`	Input vertices and undirected edges
`N = n + m`	Represented input size used for complexity fits
`k`	Size of a designated small vertex cover/core
`L`, `B`, `Q`	Original verifier-tree leaves, full-branching-tree vertices, directed path queries
`T(KKT)/T(A)`	Matched runtime ratio; below 1 favors KKT

For a fixed-density sparse family, `m = θ(n)`, so linear in N is also linear in n. For a complete graph, `m = θ(n²)`: a linear-in-input algorithm necessarily takes quadratic work in the vertex count. Calling both experiments “O(n)” obscures what is being measured.

The comparison starts from a tuple edge list. That is a useful public API boundary, but a program already holding a tuned adjacency representation could obtain a different Prim ranking. Algorithm names alone do not specify a practical implementation.

2. The old implementation broke local accounting

The measured original is `693c6a5`. It already followed the recognizable KKT outline. The problem was that several local operations did not have the cost required by that outline.

The clearest defect was **edge tracking sized by original IDs**. Let `R = 1 + max(original edge ID)`. Contractions initialized R-sized bitmaps; recursive calls allocated result storage of that size; result merging scanned the same range. Children inherited the original range even when their own graphs had become tiny.

For compact original IDs, `R ≈ m_root`. A child with 12 edges could still pay for bookkeeping sized to millions of original edges. With sparse IDs, R was not even bounded by the graph's represented size. Negative IDs also made direct indexing invalid; computing `INT_MAX+1` in an int was another boundary defect.

This is stronger than “a slow container.” The recurrence requires local work proportional to the **current** subproblem. Repeated root-scale scans invalidate that premise. A hierarchical weighted path makes the issue concrete: its surviving main branch needs Θ(log n) contractions, and a tree has no F-heavy edges to remove. Scanning an original dense-ID range R≈n at those levels contributes Ω(n log n) bookkeeping. Sparse IDs can make R even larger without adding any graph edges.

`60c65b4` changed accumulation to selected-ID vectors and local tracking to current edge positions. Original IDs became payload, rather than capacities. Legacy maximum-ID metadata was widened and ceased controlling algorithm storage.

Engineering lesson: recursive data structures must shrink with recursive inputs. Replacing a hash set with a bitmap is beneficial only when the bitmap's domain is appropriate.

3. A lookup table was not linear to construct

The verifier uses a full branching tree and a lookup table over depth masks. A small final table does not imply cheap preprocessing.

In the old median-table builder, repeated subset enumeration rebuilt related information many times. Consider only its outer iteration `s=h`, where h is the branching-tree height. Across all subsets of an h-element set, the total number of included elements is `h·2^(h−1)`. The old code explicitly visited those included bits. That slice alone therefore costs `Ω(h·2^h)`.

A full binary branching tree can have `L=2^h` original leaves. The lower bound becomes `Ω(L log L)`, before counting the other repeated enumeration. This was a genuine asymptotic defect in preprocessing, not merely recursion overhead.

`312954e` replaced enumeration with a recurrence: remove the highest and lowest set bits and reuse the already-computed median for the inner mask. Every nonzero mask takes constant word operations.

Why does the resulting table stay linear? In this particular reduction, every original leaf receives one parent per round, all leaves have the same depth, and every internal node has at least two children. Thus `L ≥ 2^h`. The `2^(h+1)−1` nonzero masks require fewer than `2L` nonzero-mask iterations (and at most `2L` allocated entries including zero).

The equal-depth condition matters. An arbitrary tree in which every internal node has two children can still be highly unbalanced; that property alone would not justify logarithmic height. The current source and regression invariants rely on the stronger structure produced by the reduction.

The source-level repair is in median_table.hpp. Its tests check the relationship between preprocessing work and valid branching-tree size; a timing fit is unnecessary to detect this class of error.

4. Randomness, correctness and the old benchmark

A third issue concerned the sampling argument. Each old recursive sample called `srand(seed)` again with the same seed. That replayed masks and changed process-global RNG state. The standard sampling analysis assumes fresh suitable random choices conditional on the current subproblem.

`ccb833f` introduced one private, continuing `mt19937` stream per invocation. This removes repeated RNG prefixes at each recursive sample and caller interference; replaying an entire invocation with the same seed remains intentional. A finite PRNG is still not a mathematical source of independent random bits; the usual randomized model remains an assumption. Reusing a seed was a problem for the complexity argument, **not evidence by itself that the resulting MST was wrong**.

The earlier benchmark also asked a narrower question than its curve suggested. It fixed `n=10,000` and increased m from 1.25 million to 20 million. That was primarily a density sweep, not a vertex-scaling experiment. Generation reused a default seed, measured outputs were not independently validated, and KKT input conversion sat outside its loop. Those facts restrict interpretation; they do not make every recorded duration meaningless.

Additional repairs fixed a Prim/KKT global `Edge` type collision when linked together, made test iterations generate distinct seeded graphs, compared exact unique MST edge sets, and made the local build reproducible. These were correctness or evidence-quality repairs, not KKT speedups.

The present benchmark certifies a reference tree using source-edge membership, connectivity/acyclicity and the cycle-optimality property, independently of the measured algorithms' logic. Every timed result and warmup must match that unique MST. Exact IDs avoid relying on the legacy `int`-returning cost helper, which can overflow on large sums.

5. Then we reduced the constants

After the structural repairs, the practical series began from `0820435`. Six retained changes improved the implementation without changing graph distributions or switching algorithms.

Repair	Avoided work	Preserved invariant
`4eb3ca1`: local heavy-edge marks	Internal heavy-edge hash sets and per-edge membership probes	Each mark refers to a position in the current graph
`2749ced`: contiguous path-query ranges	Linked query lists and duplicate metadata	Same ancestor queries and maxima
`2e37b4e`: cheapest-neighbor walks	Selected-edge adjacency and an m-sized duplicate bitmap	Same selected IDs and component numbering
`af12309`: temporary parent-position IDs	Hash-based sampled-forest recovery	Caller IDs and RNG draws preserved
`ba39977`: consume temporary vectors	Extra graph copies during isolate removal	Edge order, weight and ID preserved
`2d4fbf1`: reserve final ID-set buckets	Repeated bucket growth	Same output membership

The first three structural fixes and these six constant-factor changes are different categories. Flat storage can reduce allocations and improve locality while remaining the same asymptotic algorithm. Its benefit must be measured, not inferred from Big-O.

One proposed optimization was **rejected**. Caching each minimum edge's cost beside its index enlarged the per-vertex working set from four to eight bytes. It produced small gains in some cases and a roughly 6% regression on the largest tested `random_32` case. The current code retains only indices; the rejected patch and all results remain in `db0717e`.

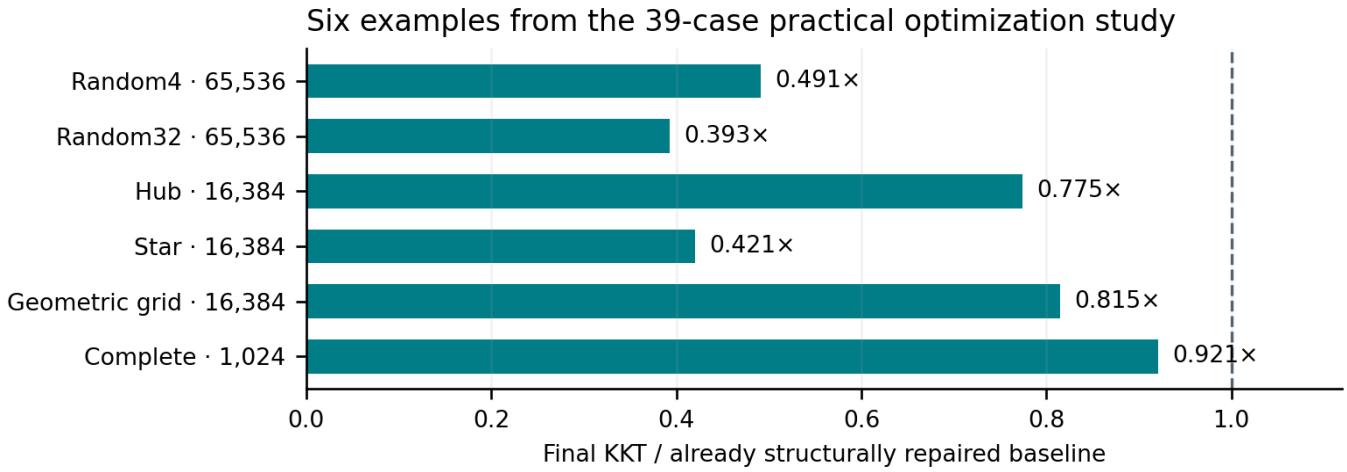
Reserving the final hash table does not eliminate its node allocations. The public output representation remains a meaningful constant, especially when the graph collapses almost immediately.

6. How much did those repairs matter?

Two experiments answer different historical questions. They must not be multiplied into a synthetic end-to-end speedup: their inputs, timing boundaries and observation structures differ.

Original repair example	Original KKT	Repaired KKT	Matched speedup
n=1,000; m=400,000	16.52 ms	5.07 ms	3.28×
n=4,096; m=16,384	3.38 ms	1.35 ms	2.48×
n=4,096; m=131,072	17.58 ms	5.18 ms	3.32×
n=4,096; m=524,288	62.66 ms	13.99 ms	4.54×
n=16,384; m=524,288	131.36 ms	42.34 ms	3.12×

The first repair study measured 13 cases, with 27 observations per case, phase and algorithm. Its original-to-repaired timings excluded input construction. The practical study used the later common tuple-input boundary, 39 cases, three graph seeds, three KKT seeds and four repetitions. The complete results preserve cases whose individual trials or intermediate optimization stages became slower.



The practical primary campaign improved the KKT case-level paired median on all 39 cases, with final/baseline ratios approximately 0.393–0.969. That statement is about those case medians, not every graph or trial. Dense complete graphs in the separate large campaign still regressed relative to its historical baseline. Making KKT much faster did not make it universally fastest.

7. What “expected linear” requires now

The original KKT paper establishes a randomized linear-time MST algorithm in a unit-cost comparison RAM. Its key sampling lemma bounds the expected number of F-light edges by `n/p`, where F is the minimum spanning forest of an independently sampled p-subgraph. The implementation uses `p=1/2`, two initial Borůvka contractions, and a linear-time heavy-edge classifier.

A compact way to check the implementation's intended accounting is to assign each recursive input the potential `Φ = m + 2n`. After two contractions, its vertex count is at most `n/4`. The sampled child has at most m/2 edges in expectation; the light-edge child has at most n/2 in expectation. Together their vertex counts are at most n/2. Therefore:

`E[Σ Φ(children) | current input] ≤ m/2 + 3n/2 ≤ (3/4)·Φ(input)`.

If every local phase costs at most `C·Φ`, expected total local work is bounded by a geometric series, at most `4C·Φ(root)` in this accounting. This is a source-audit explanation under the stated model, not a formal verification of every C++ path. It also shows exactly why root-ID-sized bookkeeping was unacceptable: no constant C could tie that work to an arbitrarily small current input.

For explicitly represented isolated vertices, `O(n+m)` is the sensible claim. Hash output construction assumes average/expected constant-cost operations; integer counts, masks and memory must be representable. The current benchmark domain has connected simple graphs, distinct integer weights and unique edge IDs. Arbitrary tied weights and invalid verifier forests are outside what this evidence establishes.

8. Auditing the verifier's less obvious costs

The recursive shape is only half the story. A heavy-edge classifier with a hidden logarithmic factor would change the bound.

Phase	Current accounting	Practical cost still present
Contraction	At most four current-edge visits and a constant number of vertex passes; stamped buckets merge parallel edges	Multiple arrays and full scans
Forest decomposition	Each forest arc, vertex and within-component query edge is grouped once	Component metadata and materialization
Full branching reduction	Active trees shrink geometrically; fewer than `2L` output vertices	Intermediate arrays and hierarchy construction
Median masks	Constant work per mask; fewer than `2L` masks	Table initialization
LCA preprocessing	Block sparse table and ±1 microtables have linear aggregate size	Nested allocations and nonuniform small-input constants
Path queries	Aggregate comparison/search bound, not constant work for each individual search	Query storage, bit operations and branch behavior

For LCA, an Euler tour has length E. Blocks of roughly $(\log_2 E)/2$ give $O(E/\log E)$ block minima, whose sparse table costs O(E). At most $2^{(b-1)}$ block masks each need O(b²) work; $O(\sqrt{E}\cdot\log^2 E)$ is O(E) asymptotically. Small finite ranges can still have awkward constants.

For the path-search bound, keep notation precise. Komlós's verification analysis uses **B full-tree vertices**, not L original leaves: $3B + B\cdot\log_2((Q+B)/B)$ comparisons. This is O(B+Q). Hagerup's tree-path-maxima implementation addresses implementation overhead as well. The current audit checked the code's structural correspondence; it did not formally prove equivalence to all details of those algorithms.

No obvious additional asymptotic factor remains in the audited phases. That conclusion is more defensible than claiming that a fitted timing curve proves the implementation linear.

9. GoogleTest is not a complexity estimator

GoogleTest runs correctness and structural regression tests. Here those include local scan bounds, sparse-ID counter invariance, collector scoping, star termination, geometric forest-reduction work and median-table size bounds. Those tests can detect a return of the known defects. They do not infer a wall-clock exponent.

Google Benchmark provides the requested fitting feature. The harness calls `SetComplexityN(n+m)` and `Complexity(benchmark::oAuto)`, using manual steady-clock intervals. Each family and graph seed gets its own registration. Families of different density are never pooled into one fit.

There is an important version-specific detail. The repository pins **Google Benchmark 1.5.2**. Its actual implementation chooses the automatic model using CPU time, then fits manual/real time to that selected model. `UseManualTime` does not change the clock used to select the label in that version. The model is a zero-intercept least-squares fit; its printed RMS is normalized by the mean observed cost.

The library searches a small menu: constant, log N, N, N log N, N^2 and N^3. An N^2 label over a short range does not prove quadratic complexity, and an N label does not certify linearity. Mixtures of costs, setup intercepts, changing graph reduction, memory behavior and noise can favor the wrong asymptotic-looking label.

The analysis therefore retains the built-in labels **and** independently fits exported manual times. It reports `N`, `N log N`, `N(log N)²`, `N²`, and a free power `cN^p`, with full-window and upper-half fits and residuals. The current user guide describes the API; the pinned source determines this run's semantics.

A free-power fit is also a finite summary. Its upper-half window emphasizes the largest five timing sizes, but it is still only five points. The four-size work sweep has only two points in its upper half, so that value is simply a two-point slope.

Google Benchmark registrations run in fixed order and do not use the rotating paired schedule of the main comparison. Its algorithm ratios are ratios of arithmetic mean times. The main campaign instead aggregates matched trial ratios with hierarchical medians. Those summaries answer related but different questions and are not pooled.

10. Does the current implementation measure as O(N)?

There is no single affirmative answer across families. The earlier final-source diagnostics found approximately linear behavior on stars, but visibly superlinear finite-range timing and rising work per input on random graphs.

Earlier final-source family	KKT full-range timing p	Upper-half p	Counted work/N, smallest → largest
Random, `m=4n`	1.175	1.120	17.69 → 31.38
Random, `m=32n`	1.379	1.527	11.80 → 36.12
Star	1.001	1.024	6.001 → 6.000
Hierarchical path	No timing fit	No timing fit	36.35 → 36.00

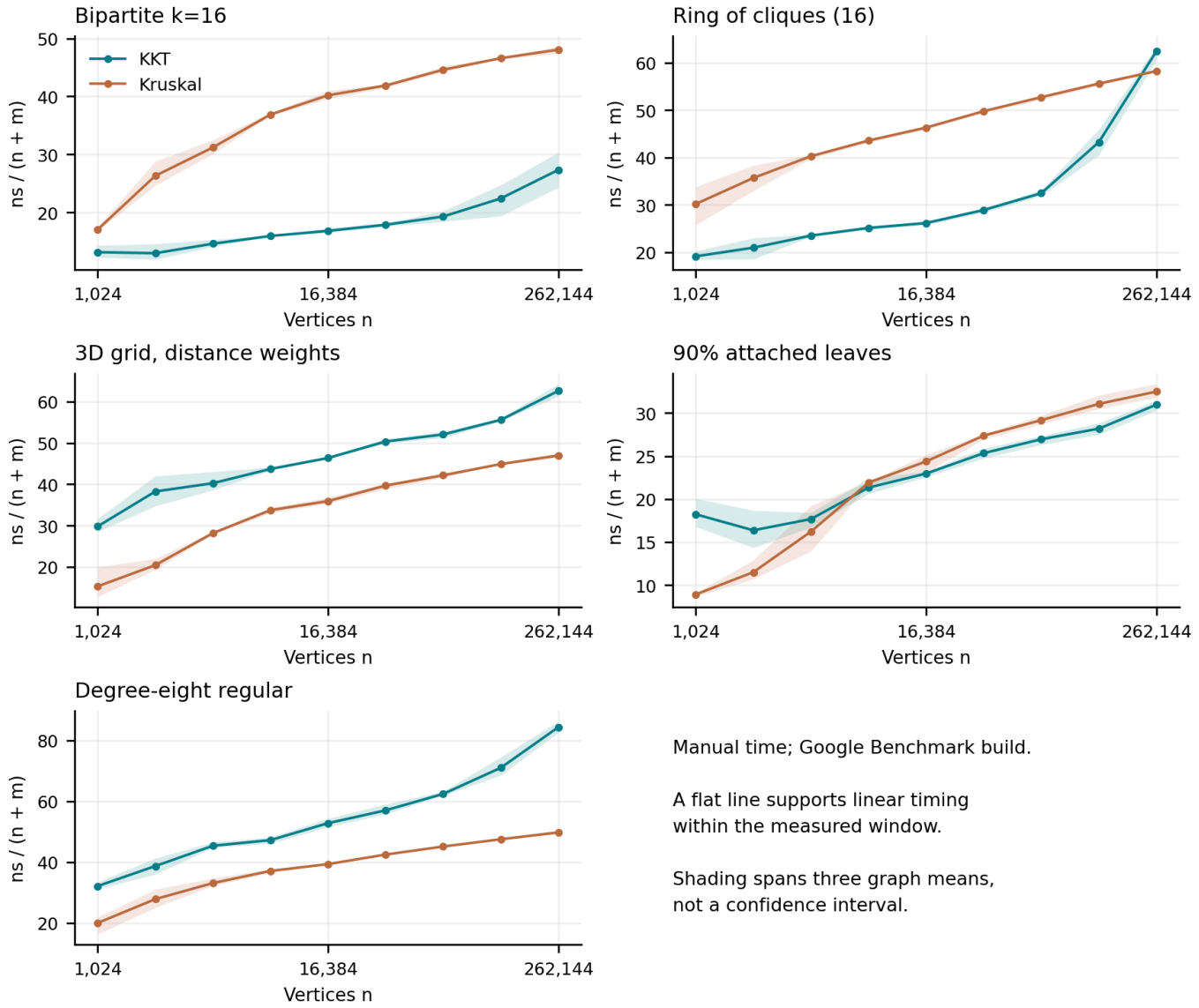
These are preserved final-source diagnostics: timing uses seven sizes `n=1,024…65,536`; work uses five sizes through 262,144. The built-in KKT labels were N for all three star seeds, N log N for sparse random, and N² for dense random. That disagreement with the abstract bound is a reason to inspect mechanisms, not to reinterpret a fit as a proof.

New final-source diagnostics:

Family	KKT full p	Upper p	Work/N, first → last	Built-in KKT label
Bipartite k=16	1.126	1.173	7.31 → 7.20	N log N (3/3)
Ring of cliques (16)	1.185	1.309	9.19 → 9.15	N² (3/3)
3D grid, distance	1.113	1.100	17.66 → 18.50	N log N (3/3)
90% attached leaves	1.115	1.102	11.43 → 14.05	N log N (3/3)
Degree-eight regular	1.157	1.167	24.68 → 32.45	N log N (3/3)

The new fits use nine doubling sizes through 262,144, five preselected mechanisms, two algorithms, three graph seeds and three batches of three calls: **810 raw timing records representing 2,430 calls**. They use Bazel `-c opt --copt=-O3 --copt=-DNDEBUG`, separately from the native `-march=native` comparison campaigns. Sampling seeds and repeated batches do not create additional independent graphs.

11. See where linear timing stops being a good description



A flat `time/N` curve supports linear timing *over this window*. A rising curve means cost grows faster than N over the observed range. The shaded ranges span the three graph means; they are descriptive ranges, not confidence intervals.

All five new KKT timing curves rise overall in time per represented input. Full-window exponents range from **1.113 to 1.185**; none receives the built-in N label. The clique case receives N² despite a free-power exponent of only 1.185. That is a particularly clear warning against reading the selected label as a mathematical classification.

The fixed-cover and clique work proxies are nearly flat per N (7.31→7.20 and 9.19→9.15) while their timings rise. Changing counted algorithmic work alone cannot explain those curves. Memory/allocator effects and uncounted operations are plausible contributors, but these runs do not profile them. The regular and leaf-core proxies also increase, showing that finite-size work mix matters.

This diagnostic is less controlled than a dedicated benchmark machine: the startup log records load averages 18.42, 16.06 and 14.13. Those are system-load measures, not CPU-utilization percentages. Treat small fit differences and near-crossovers cautiously; retain the raw records instead of smoothing them away.

12. Why Big-O does not give us a practical crossover

If two exact cost models were `aN` and `bN log N`, with stable positive constants and an unlimited domain, the linear one would eventually win. But Big-O **upper bounds alone** do not establish those exact models or a usable crossover. Even in the idealized model, the threshold grows exponentially with `a/b` when natural logarithms are used.

KKT performs contraction scans, samples, recursion, verifier preprocessing, path queries, graph materialization and output construction. Kruskal mostly performs a contiguous sort and union-find scan. Several linear passes with allocation and indirect accesses can cost more than an optimized sort over the sizes the machine can hold.

The work counters explain another finite-range effect. A small dense graph may collapse almost completely during contraction; as n grows at fixed average degree, more distinct component pairs remain and fewer parallel edges disappear. Thus the amount of recursion and verification **per original input edge** can rise before any asymptotic envelope becomes visible. In the earlier random32 sweep, root retention after two rounds grew from about 4.2% to 87.4%.

The counters are a deliberately limited proxy. They count visits, explicit initializations, table writes, edge materializations and hash API items. They do not count allocator internals, implicit vector growth copies, deallocation, hash collisions, cache misses or CPU instructions. We can observe changing work mix and time per proxy event; we cannot name a hardware bottleneck as measured fact without profiling it.

The correct practical question is therefore: **which inputs get cheap, substantial reduction before expensive recursive work, and can a simpler competitor exploit the same reduction?** The next experiments test exactly that.

13. The additional common families

The new matrix contains **52 cases: 13 model variants at four sizes**, $n=\{4{,}096, 16{,}384, 65{,}536, 262{,}144\}$. Eleven are new; hub and random4 are unchanged controls. These are explicitly defined synthetic models, not claims to reproduce every application graph with the same informal name.

Group	Variants and edge count	Question tested
Unbalanced bipartite	Small side k=16,256,$\lfloor\sqrt{n}\rfloor$; every peripheral degree 4; $m=4(n-k)$	Does a small cover force useful contraction as its size grows?
Leaf-rich sparse core	Core $c=\lfloor n/2\rfloor$ or $\lfloor n/10\rfloor$; core has 4c edges; $m=n+3c$	How much do forced leaf edges help when a hard core remains?
Ring of cliques	16-vertex cliques linked in a cycle; $m=121n/16$	Does strong local redundancy help without artificially cheap local edges?
Open 2D mesh	Four-neighbor grid, random versus distance ranks	Same topology, different weight correlation
Open 3D mesh	Six-neighbor near-cubic grid, same paired weighting	Higher local connectivity and physical geometry
Degree-eight regular	Protected Hamiltonian cycle plus degree-preserving switches; $m=4n$	Homogeneous negative control for the hub hypothesis
Existing controls	Hub and random4, both $m=4n$	Preserve continuity with earlier studies

New nonspatial families receive a global permutation of distinct weights $1\ldots m$; no leaf, bridge, core or spanning-tree edge is deliberately made cheap. Labels and edge order are shuffled independently of topology. Distance variants rank exact squared geometric lengths, breaking ties by original edge ID; their paired random variants share every other input attribute.

The regular generator is connected and exactly degree eight. It is **not** claimed to be a uniform random-regular draw, a rapidly mixed sample, or a proven expander. Likewise the meshes are not nearest-neighbor graphs of independently scattered points and are not road-network datasets.

14. Protocol fixed before the results

The pre-timing plan fixes the families, sizes, diagnostics and reporting rules. This is exploratory follow-up motivated by the earlier shape study; it is not an independent confirmation using newly chosen graph seeds.

Each of two complete campaigns contains **7,488 timed outputs and 1,872 checked warmups**: 52 cases × three graph seeds × three KKT seeds × four repetitions × four algorithms. Every algorithm occupies each timed position equally within a graph/seed block. Case order is shuffled once and reused. A separate instrumented campaign contains 468 observations from 156 case/seed combinations.

Generation, certification, output checks and destruction of the returned result are outside timing. KKT's tuple conversion and temporary-input destruction are inside timing. Every algorithm includes its own result construction. No instrumented execution is used as a runtime observation.

Timing summaries take medians across repetitions, then sampling seeds within a graph, then graph seeds. Ratios are formed **inside matched trial blocks before aggregation**. A ratio of displayed median milliseconds need not equal the displayed matched ratio. Graph min/max ranges describe three graphs; they are not confidence intervals or a significance test. Repeated calls and sampling seeds are not additional graphs.

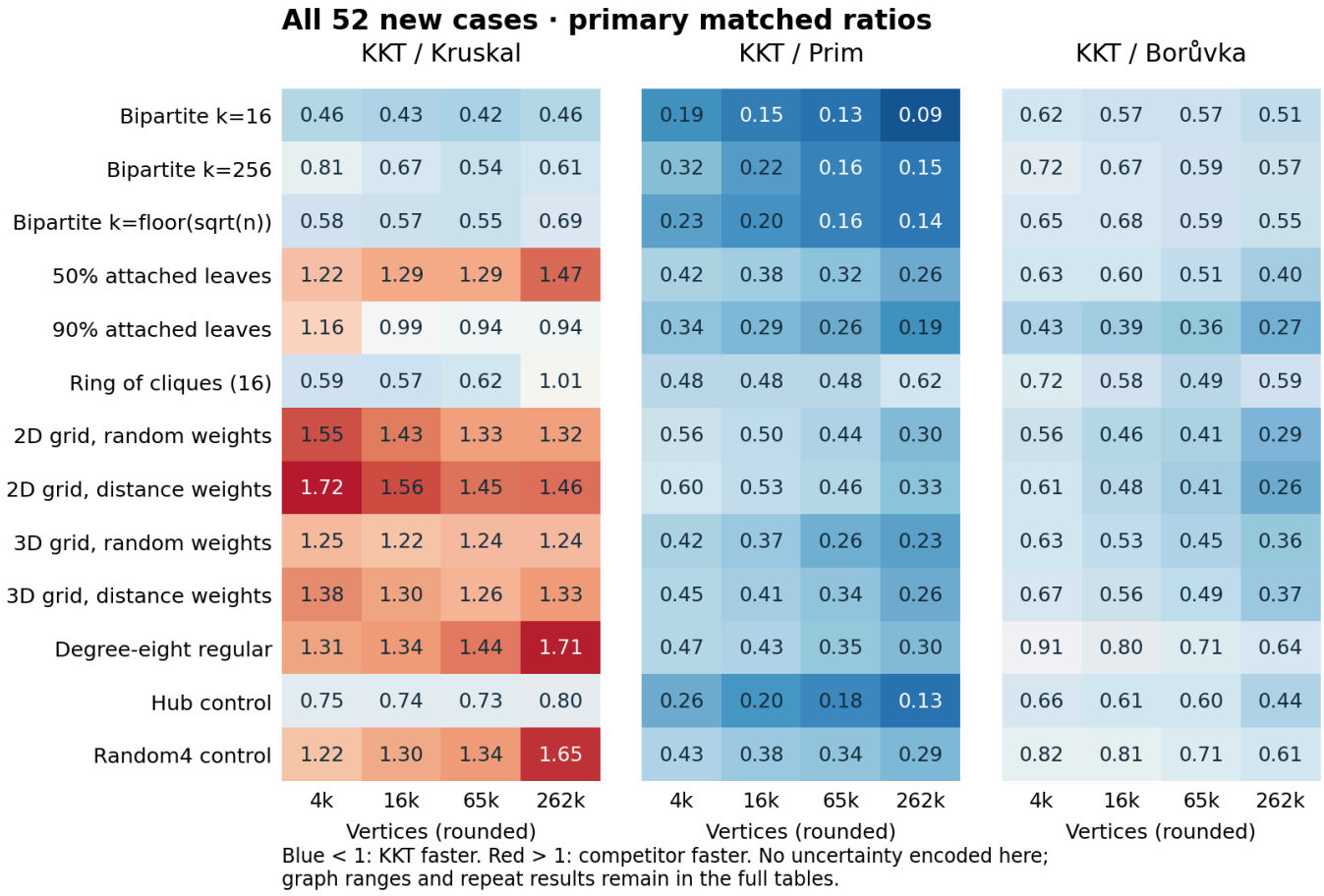
The two campaigns remain separate. A win that repeats on all three graph medians is stronger descriptive evidence than a single median crossing 1, but it still does not establish application-wide dominance. All losing cases remain in the tables and raw files.

Builds and tests finished before timing; analysis and rendering ran outside the timed phases. Only one timed process ran at a time. This is an ordinary desktop with no core affinity or controlled thermal/power state. Native comparisons use Apple clang 15, `-O3 -march=native`, assertions enabled, on an Apple M3 Max with 128 GiB RAM.

15. All new families, all three competitors

KKT leads all three original comparators by case medians in **22/52 primary cases and 21/52 repeat cases**. The same 21 cases lead on every graph median in both campaigns: all 12 bipartite parameter/size cases, all four hub controls, the three smaller clique cases, and the two larger 90%-leaf cases.

KKT beats Prim and ordinary Borůvka by paired case medians in all 52 cases in both runs. Individual observations still lose: primary has 0/1 losses to Prim/Borůvka; repeat has 18/35, out of 1,872 matched pairs per comparator. Against Kruskal, there are 1,085 / 1,091 individual losses. The result is strong structural selectivity, not universal dominance.



Ratios below 1 favor KKT. The repeat campaign is retained separately; the appendix gives both runs. Near-ties are described as such rather than converted into broad algorithm recommendations.

16. The largest new instances

At `n=262,144`, densities vary by model. Comparisons are within a graph; the table is not a fixed-m contest between graph shapes. Milliseconds below are primary hierarchical medians. The final column compares KKT with the fastest competing case-median implementation, using its matched ratio in each campaign.

Family	KKT ms	Kruskal	Prim	Borůvka	Matched ratio: primary / repeat
Bipartite k=16	28.54	62.00	290.42	55.56	0.514 / 0.512 (Borůvka)
Bipartite k=256	37.93	62.50	263.93	67.16	0.606 / 0.672 (Kruskal)
Bipartite k=⌊√n⌋	42.97	62.03	301.83	73.26	0.689 / 0.690 (Kruskal)
50% attached leaves	59.04	40.25	218.76	140.47	1.469 / 1.373 (Kruskal)
90% attached leaves	18.89	19.83	101.53	69.20	0.943 / 0.953 (Kruskal)
Ring of cliques (16)	129.95	129.06	206.91	222.74	1.006 / 1.051 (Kruskal)
2D grid, random	43.94	33.39	145.82	155.17	1.320 / 1.340 (Kruskal)
2D grid, distance	47.97	33.44	150.92	187.64	1.459 / 1.505 (Kruskal)
3D grid, random	59.61	48.07	268.00	173.73	1.242 / 1.300 (Kruskal)
3D grid, distance	64.51	49.01	254.98	175.78	1.334 / 1.325 (Kruskal)
Degree-eight regular	110.06	64.61	366.63	179.52	1.712 / 1.815 (Kruskal)
Hub control	51.43	64.29	381.75	117.01	0.799 / 0.871 (Kruskal)
Random4 control	107.36	64.72	367.06	167.88	1.647 / 1.681 (Kruskal)

The fixed k=16 bipartite family is about **2.16× / 2.10× faster than Kruskal** by the reciprocal matched ratio. Growing the cover reduces the advantage. The 90%-leaf gain is only about 5–6%; the clique family has crossed from a smaller-size lead to a largest-size near-tie/loss. None of the four spatial variants beats Kruskal at this size.

The labels identify the model parameters, not a claim that every graph with leaves, cliques, geometry or hubs will behave similarly. In particular, a fixed small cover and a cover that grows with n are different asymptotic families.

17. Why a genuinely small core helps

The useful structural property is a **small vertex cover**, not simply a large maximum degree. Suppose a connected graph has a cover C of size k: every edge touches C. Vertices outside C have no edges between them. Each chooses its cheapest edge to C in the first Borůvka selection. Every selected-edge component therefore contains a cover vertex, so at most k components remain.

After loops and parallel edges are removed:

$n_1 \le k$, and $m_1 \le \min(m - n + k, k(k-1)/2)$.

The edge-count term also uses the selected forest's at least $n-k$ edges. A second contraction leaves at most $\lfloor k/2 \rfloor$ vertices. This argument is independent of the weight order, as long as the selection consistently produces the safe forest assumed here.

For fixed $k=16$, the first residual has at most 16 vertices and 120 edges, regardless of how many peripheral vertices were present. The remaining randomized part is then tiny. For $k=256$, the corresponding bound is 32,640 edges; at smaller n it can be loose. For $k=\lfloor\sqrt{n}\rfloor$, the generic first residual edge bound is O(n), not a constant-size tail. A growing core can therefore consume much more of the saved work.

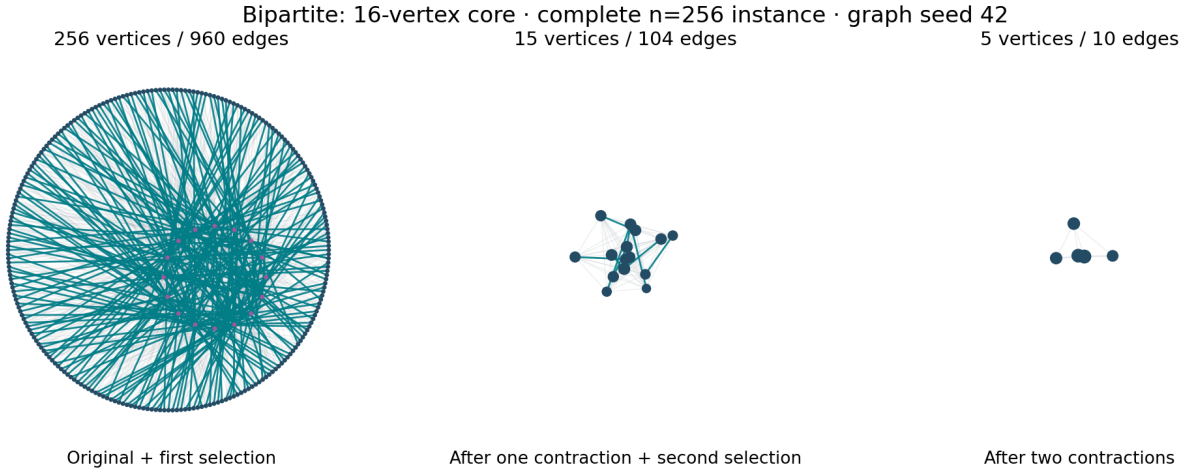
At n=262,144	KKT/Kruskal: primary / repeat	Root edges remaining after two rounds
Bipartite k=16	0.463 / 0.475	0.001049%
Bipartite k=256	0.606 / 0.672	0.180930%
Bipartite k=⌊√n⌋	0.689 / 0.690	0.757011%

The residual fractions are arithmetic means over the separate work runs. They make the mechanism concrete: the fixed 16-vertex cover almost eliminates the recursive problem, while the growing cover leaves a larger tail. They are not a measured timing decomposition.

At $n=65{,}536$, the k=256 and k=⌊√n⌋ generators produce **identical seeded weighted inputs**. Their separate timing observations are useful repeatability data at that parameter intersection, but are not independent evidence for two different topologies.

An appropriately implemented Borůvka can exploit the same bound. The property does not belong uniquely to KKT's randomized step; the ablation later measures that distinction.

18. Watch the core disappear



This full 256-vertex instance is generated by the same bipartite16 model, graph seed 42. The production first round leaves 15 vertices and 104 edges; the second leaves five vertices and ten edges. These are actual exported outputs, checked against an independent disjoint-set reconstruction and minimum-parallel-edge reduction.

Teal edges in the first two panels are selected safe edges. Purple vertices identify the designated 16-vertex core. Supervertex positions are **centroids of their original member vertices**, with a common scale across stages; the drawing does not silently spread a tiny residual back out to fill the page. Node size increases with the number of original members.

The small instance illustrates the mechanism. It is not substituted for a timed large instance, and its residual counts should not be quoted as the 262,144-vertex case's counts.

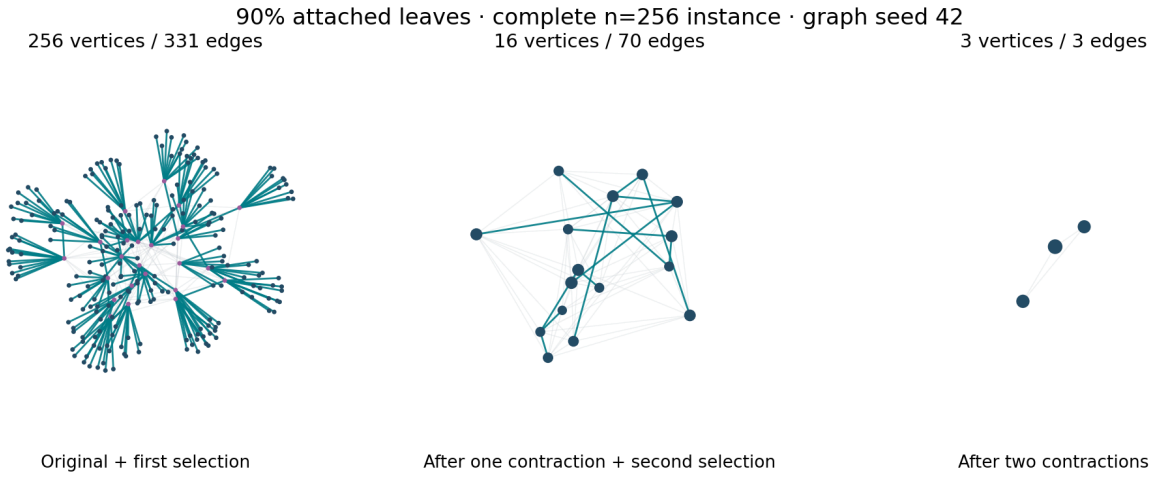
Open the interactive 3D explorer to rotate this graph, inspect member counts, step through contraction, and view the sampled forest and removable heavy edges. The explorer includes seven full graphs, including a regular negative control. Nothing is downsampled from a large graph and presented as the full input.

The output verifier's heavy-edge labels are independently checked by explicit maximum-edge searches on forest paths. Edges between forest components remain light because no path exists. The visual therefore illustrates the actual classification, not a guessed coloring.

19. Leaves: cheap output, but a residual problem

Every degree-one vertex forces its incident edge into the MST. KKT's first selection accepts those edges regardless of their weights. A star finishes immediately: every edge is selected, one component remains, and there is no sampling or verification.

A graph with many leaves is different. The attached leaves vanish, but the random sparse core still requires an MST. For a core of c vertices with 4c edges and `n−c` attached leaves, the graph has `m=n+3c`. A 90% attached-leaf model leaves a much smaller core than a 50% model, but both must still read the original input and construct roughly n output IDs.



At 262,144 vertices the 90%-attachment model takes **0.943 / 0.953× Kruskal**, while the 50% model takes **1.469 / 1.373×**. The smaller core is enough to change the practical ranking. At 16,384 vertices, however, the 90% case is only 0.987 / 1.005×: that apparent first-run crossover does not repeat.

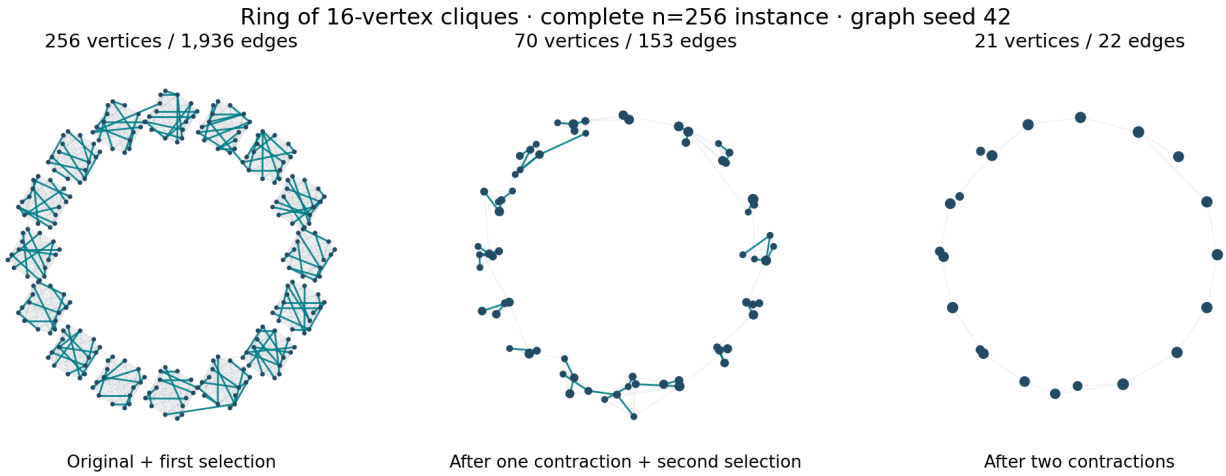
After two root rounds, the 90% model retains about 23.59% of its original edges; the 50% model retains 60.24%. Neither resembles the star's immediate termination. This is why “many leaves” can help without guaranteeing that KKT wins.

The “90%” name is rounded: `c=⌊n/10⌋`. The designated attachments are exactly leaves, while core vertices can also have degree one. The reported measured leaf count comes from the actual graph rather than the nominal percentage.

The earlier balanced binary tree is a useful counterexample to “trees favor KKT.” At 65,536 vertices it took roughly 1.49–1.50 times Kruskal's time despite substantial contraction. Kruskal sorts only `n−1` edges there; KKT still pays repeated vertex passes and output costs. Tree shape and termination depth matter.

20. Small dense communities without planted weights

A ring of cliques separates two effects. Each 16-vertex clique contains 120 internal edges, providing substantial local redundancy. Only one edge joins each consecutive pair of cliques around a cycle. Internal and inter-clique edges share the same global random weight assignment.



The inter-clique connections are **not individually forced bridges**: contracting each clique turns them into a cycle. With globally mixed weights, an MST may keep every inter-clique connection and instead omit an internal edge on the corresponding original cycle. We cannot assume that solving each clique independently and dropping one external connection gives the global MST. This generator also avoids making all local edges artificially cheap.

KKT beats all original comparators on every graph median at 4,096, 16,384 and 65,536 vertices in both campaigns. At 262,144 it takes **1.006 / 1.051× Kruskal**, losing that lead. Root contraction is still excellent: only **0.975%** of original edges remain after two rounds at the largest size.

The counted work stays near 9.15 events per N, while time per N rises. Moreover, among verifier-input edges the mean retained-light fraction is about **99.69%** in the largest work case: filtering removes little from the already sparse tails. Excellent initial contraction therefore need not imply a benefit from the randomized continuation.

This family tests bounded-size dense neighborhoods repeated as n grows. It differs from the previous model with a fixed number of 16 communities whose individual sizes grow. “Community structure” alone therefore does not predict the same scaling: the size and number of communities, the number of external edges, and weight correlations all change the residual problem.

The figure retains all 256 vertices and all 1,936 original edges. Its first contraction leaves 70 vertices and 153 edges, and the second 21 vertices and 22 edges. Near-tree residuals also explain why simply continuing contraction could be inexpensive; that hypothesis is tested rather than credited automatically to sampling.

21. Geometry and edge weights are separate variables

The paired meshes are the most controlled new weight experiment. At each n and graph seed, random and distance variants have identical endpoints, IDs, labels, input order and jittered coordinates. Only their weight ranking differs. Distance ranks preserve exact integer squared-distance order, with deterministic tie-breaking. The minimum spanning tree is therefore the one for those distances under that tie order.

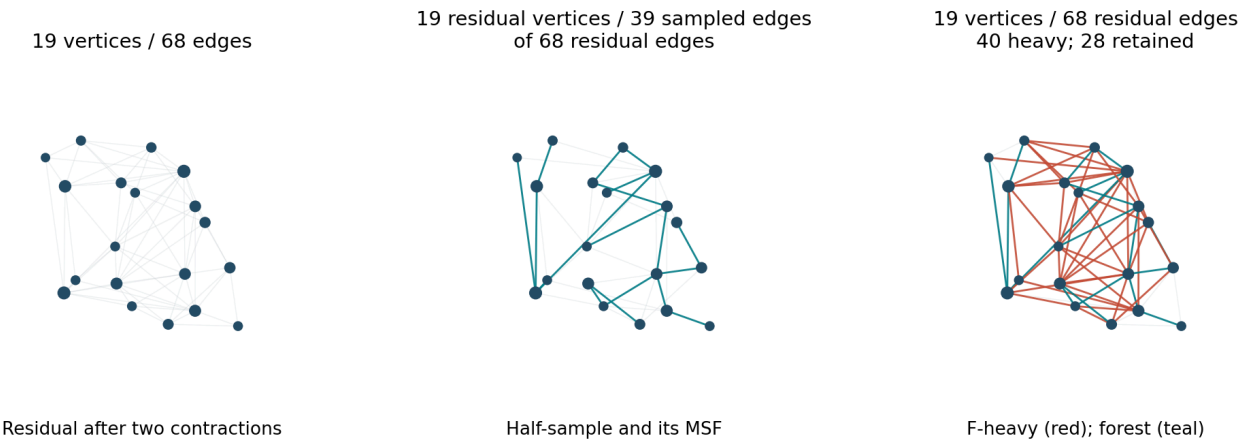
The 2D meshes use open four-neighbor square grids. The 3D meshes use open six-neighbor dyadic grids: 16^3, $32\times32\times16$, $64\times32\times32$, and 64^3 at the four main sizes. Active coordinates are lattice positions plus bounded jitter; geometric proximity does not create extra edges.

All four mesh variants lose to Kruskal at all four main sizes in both campaigns. At 262,144 vertices, paired KKT/Kruskal ratios range from **1.242 to 1.505** across variants and runs.

Distance ranking does not rescue KKT here. Matched distance/random KKT times are **1.100 / 1.140×** for the 2D meshes and **1.076 / 1.038×** for the 3D meshes at the largest size. KKT's relative ratio against Kruskal also worsens: 1.085 / 1.135× and 1.063 / 1.037×, respectively. These weight effects are specific to this paired mesh construction.

The work traces agree in direction: distance-weighted meshes leave more edges after two rounds—13.62% versus 10.75% in 2D, and 18.31% versus 15.72% in 3D. This provides a mechanism consistent with the timing; it does not separately measure the causal contribution of every later phase.

The filtering step on the same full 3D-mesh instance · sample seed 0



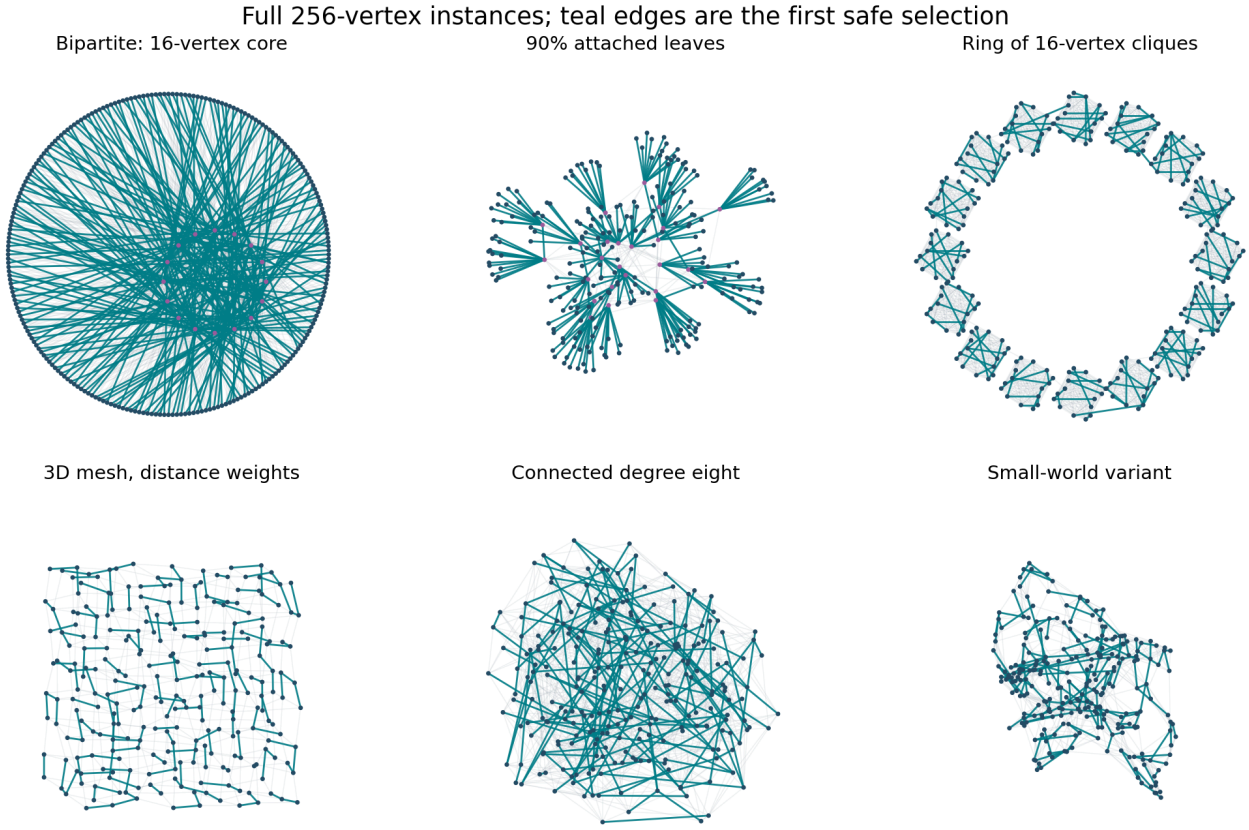
Here the residual has 19 vertices and 68 edges. A half-sample uses the same seed-zero root sampling bits as production KKT. Teal is the sample's minimum spanning forest; red edges are F-heavy. On this particular explanatory instance, 40 edges are heavy. That count demonstrates filtering activity; by itself it says nothing about whether the activity saves time compared with another continuation strategy.

These are spatial mesh workloads, not measured road networks, Delaunay graphs or k-nearest-neighbor graphs. Results should not be transferred to those application families without obtaining and validating appropriate instances.

22. Keep the counterexamples visible

At 262,144 vertices, degree-eight regular graphs take **1.712 / 1.815× Kruskal**, and random4 takes **1.647 / 1.681×**. Their two-round root residuals retain 75.88% and 75.42% of original edges. KKT then performs substantial verification and recursion after paying for the contractions.

Neither loss disappears because many edges are eventually classified as heavy. The retained-light fractions across verifier inputs are only about 3.65% and 3.31%, respectively: filtering is active and powerful, but its total cost still exceeds Kruskal's on these tests. Activity is not the same as net benefit.



The regular graph controls degree heterogeneity while remaining connected and simple. Random4 provides continuity with the older random sparse workload. Their behavior tests whether the proposed mechanism explains losses as well as wins.

Earlier shape results also constrain the story. At 65,536 vertices, preferential attachment and sparse communities lost to Kruskal by about 8–10% and 19–20%, respectively. Balanced bipartite graphs lost by 31–32%, and balanced binary trees by 49–50%. Small-world graphs showed a modest 4.6–5.6% matched advantage over Kruskal, with individual losses in both campaigns. Sixteen hubs were substantially stronger, but their 16 vertices form a fixed cover, which is a more specific property than a heavy-tailed degree distribution.

23. Does going much larger rescue general-purpose KKT?

The prior large campaign used this same final production algorithm through **1,048,576 vertices and 33,554,432 edges**. It remains relevant context; the new matrix does not replace those losses with smaller favorable inputs.

Preserved large input	KKT	Kruskal	Interpretation
Random4, n=1,048,576	893.7 ms	284.7 ms	Matched KKT/Kruskal ≈3.14
Random32, same n	10.014 s	2.385 s	Matched ratio ≈4.21
Hub, same n	358.5 ms	283.6 ms	The smaller-size KKT lead disappeared
Star, same n, focused confirmation	27.42 ms	62.37 ms	Matched ratio ≈0.442
Complete, n=8,192	4.456 s	2.328 s	Prim ≈0.432 s; Borůvka ≈0.404 s

The broad large campaign used one repetition and no warmup for all graph/sampling seed combinations; focused hub/star confirmations used four repetitions and one warmup. They are labeled separately in the retained evidence and focused confirmation. Rounded milliseconds here are case medians; matched ratios need not equal their quotient.

These results reject a practical “just increase n” strategy on the tested random families. A strong star result does not repair that generality gap. The new structural families instead ask whether there are useful shapes where large-input processing remains cheap.

24. The decisive ablation: continue compacting Borůvka

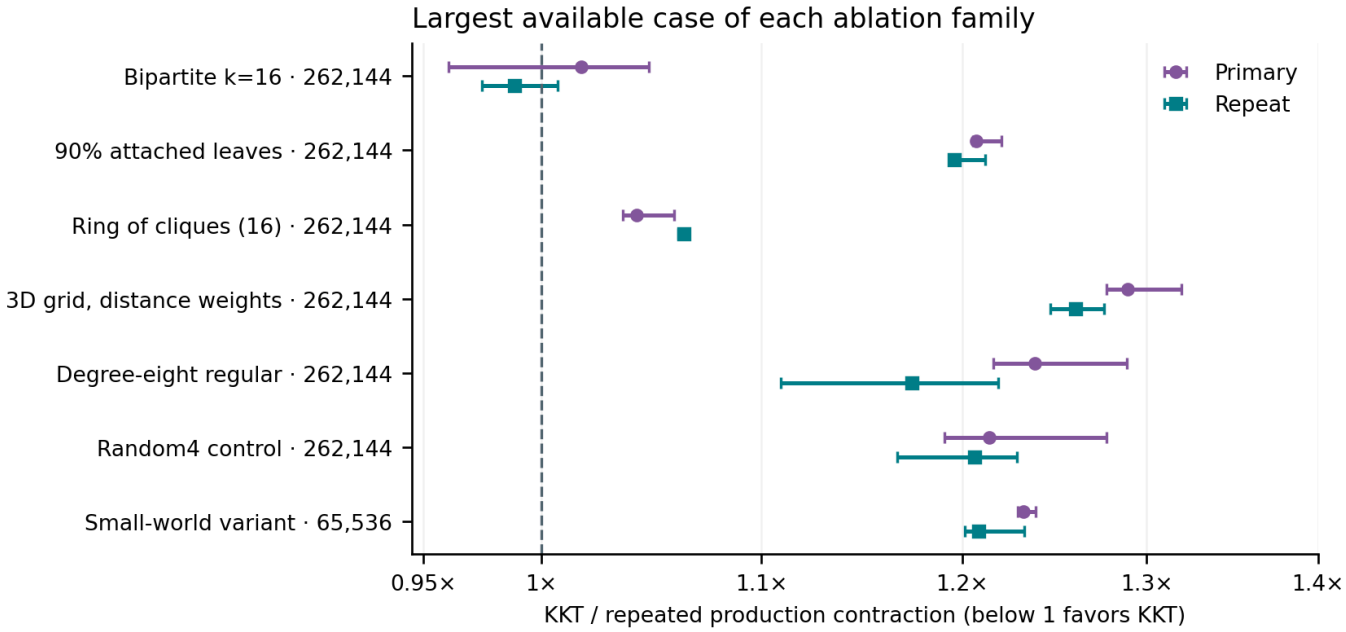
The repository's ordinary Borůvka baseline repeatedly scans the original edge set and recomputes components on the original vertices. It does not physically compact and deduplicate the graph as KKT does. Beating that implementation cannot establish a benefit from randomized sampling.

The ablation repeatedly calls **the exact production** `boruvka_step` until no edges remain, accumulates selected IDs, and constructs the same reserved ID hash set as KKT. Both start from the same tuple list and include conversion, temporary-input destruction and result construction. Every output receives the same exact-ID correctness check against a certified MST.

The fixed matrix contains 20 cases: six common families at 4,096/65,536/262,144 vertices, plus the existing small-world family at 4,096/65,536. Each of two campaigns has 1,440 timed outputs and 360 checked warmups. Sixty deterministic untimed contraction traces record the residual graph after every round.

Compacting Borůvka has the lower paired case median in **19/20 primary cases and 18/20 repeat cases**, with the same 18 losses for KKT in both. KKT's only repeated lead is bipartite16 at 65,536 vertices: **0.989 / 0.991×**, with all three graph ratios below 1 in primary but a range crossing 1 in repeat. No case gives KKT a lead on every graph in both runs. Bipartite16 at 262,144 reverses from 1.017× to 0.988×; that is another near-tie, not a stable win.

At the largest common size, KKT costs **1.289 / 1.260×** repeated contraction on the distance 3D mesh, **1.214 / 1.207×** on random4, and **1.239 / 1.174×** on regular8. Even ring cliques favor continued contraction, 1.042 / 1.064×. Their 10–11 rounds process only about **1.083m total contraction-input edges**. Many rounds can be cheap when the edge volume vanishes quickly.



This measures the **net change of continuation strategy**. It includes iterative storage release versus recursive lifetimes, RNG setup, recursion and output accumulation behavior. It does not isolate the cost of verifier instructions alone, and it is not a claim to have produced the best possible Borůvka implementation.

25. What the combined evidence supports

The original implementation was not a faithful constant-factor realization of the intended local bounds. Two audited defects could introduce superlinear work; the private RNG stream restored the intended recursive sampling discipline. The retained storage and lookup changes then reduced practical constants substantially.

The current implementation is not empirically linear across these timing windows. Stars behave nearly linearly, several structural work proxies are flat, and the source audit supports the expected O(n+m) design under its assumptions. But the new measured time exponents exceed one, and prior large random cases become less competitive. Those statements can all be true together.

KKT is useful relative to these original baselines on particular structures, but its randomized continuation has not earned the credit. Fixed small covers provide the strongest mechanism and gains. Leaf-rich cores and small dense communities help under specific parameters. The continuation experiment shows that much of this practical advantage is available to a simpler compacting Borůvka implementation.

Useful predictors are measurable properties of the early residual, rather than an informal graph label:

1. **Residual vertices and edges after contraction.** A small cover provides a strong deterministic bound; a large degree alone does not.
2. **How quickly residual edge volume keeps shrinking.** A tiny, sparse tail may make additional compacting Borůvka rounds very cheap.
3. **Verifier workload before and after filtering.** Deleting 95% of a large residual can still cost more than avoiding that verifier pass.
4. **Unavoidable input and output work.** A leaf-rich graph may eliminate search while retaining linear conversion and hash-node construction costs.
5. **The competing implementation and input representation.** A tuned Prim heap, prebuilt CSR, radix-sorting Kruskal, or compacting Borůvka could change the practical frontier.

This is a more useful diagnosis than searching for a universal “KKT-friendly” graph name. Topology, weight correlation, representation and residual size jointly determine the observed constants.

26. What I would improve next

The benchmark work does not silently alter the production algorithm to make these results look better. It identifies implementation experiments worth doing next, each with its own correctness gate and complete before/after evidence.

First, strengthen the baseline. Promote the compacting Borůvka continuation into a carefully reviewed competitor, including appropriate disconnected-input semantics. That tests whether future KKT changes improve upon the shared efficient preprocessing rather than upon an avoidably expensive old baseline.

Second, make residual handling cheaper. Profile allocations, retained buffers and path-query work on the common failures before choosing a change. A small-subproblem base case or fewer copies might lower constants, but thresholds must be selected on a training set and evaluated on withheld graph seeds and families. A hybrid should be labeled a hybrid and its asymptotic accounting rechecked.

Third, test sampling where it has a plausible cost advantage. The useful target is a residual with many expensive edges that are cheaply certifiable as heavy, while continued contraction shrinks them slowly. Tune or redesign the verifier using measured profiles; do not plant an easy weight hierarchy and call it representative.

Fourth, obtain application data. Road networks, physical meshes, geometric neighborhood graphs, and weighted infrastructure/network datasets would test whether the synthetic mechanisms occur in practice. Record preprocessing and weight semantics, preserve disconnected components where appropriate, and use independent certification. More graph seeds and a second architecture would then test generalization.

The goal remains making KKT useful. Honest losses against a stronger continuation provide concrete engineering direction and prevent us from optimizing toward a favorable benchmark name.

27. Correctness, limitations and reproducibility

The new study passed **222 fixture checks** and **all ten Bazel test targets**. Both full comparison campaigns validated all **14,976 timed outputs and 3,744 warmups**. The separate continuation campaigns validated **2,880 timed outputs and 720 warmups**. Work campaigns and traces remain untimed evidence; the new Google Benchmark sweep checked the outputs of its 2,430 timed calls.

The common analyzer's synthetic checks include six valid scenarios and 25 malformed-data rejections; the ablation analyzer has 43 malformed/math checks. Seven full graph exports pass independent contraction and heavy-edge oracles. The 18 overlapping control case/graph realizations match every common work field in the earlier archive. Counts and hashes are reconciled in VALIDATION.md.

The MST certificate is independent of the timed routines: it checks edge membership/weights, acyclic connectivity, and maximum-edge cycle optimality. Distinct weights give a unique MST, so exact edge-ID equality transfers that certificate to every measured output. The graph exporter independently checks contractions and sampled-forest path maxima as well.

This is substantial evidence over the specified domain, not a proof for arbitrary malformed inputs, repeated IDs, every tied-weight graph or every integer size. The standalone verifier assumes a valid forest. Older UBSan checks passed; ASan failed before main even for an empty control program in the earlier environment, so no ASan success is claimed here. The current new-family study did not rerun that failing runtime.

All new primary and repeat data are retained. There is no selection of the faster run, no removal of losing families, and no synthetic combination of incompatible historical protocols. Desktop noise, only three graph seeds, repeated use of those seeds across related campaigns, shared generated instances at the bipartite parameter intersection, and synthetic-model specificity remain real limits.

To reproduce: build the Make timing/work/ablation/export binaries and Bazel Google Benchmark target; run fixture and correctness checks; then run `run_campaign.py` and `run_diagnostics.py` **serially into fresh output directories**. Run the strict analyzers afterward. Exact commands, profiles, hashes and logs are in the experiment metadata and results README. Report/view generation is deliberately separate from measurement.

28. Complete new-case table: core and community models

Each cell is `primary / repeat` matched KKT-to-fastest-competing-case-median ratio. The named competitor is selected within each campaign; a changed name is shown explicitly. Below 1 favors KKT. Full pairwise comparisons to all three algorithms, graph ranges and individual loss counts remain in the linked analysis tables.

Family	4,096	16,384	65,536	262,144
Bipartite k=16	0.620 / 0.615 (Borůvka)	0.574 / 0.603 (Borůvka)	0.571 / 0.601 (Borůvka)	0.514 / 0.512 (Borůvka)
Bipartite k=256	0.814 / 0.825 (Kruskal)	0.673 / 0.678 (Kruskal / Borůvka)	0.586 / 0.602 (Borůvka)	0.606 / 0.672 (Kruskal)
Bipartite k=⌊√n⌋	0.645 / 0.645 (Borůvka)	0.676 / 0.606 (Borůvka)	0.594 / 0.575 (Borůvka / Kruskal)	0.689 / 0.690 (Kruskal)
50% attached leaves	1.222 / 1.209 (Kruskal)	1.293 / 1.271 (Kruskal)	1.292 / 1.296 (Kruskal)	1.469 / 1.373 (Kruskal)
90% attached leaves	1.164 / 1.113 (Kruskal)	0.986 / 1.005 (Kruskal)	0.938 / 0.946 (Kruskal)	0.943 / 0.953 (Kruskal)
Ring of cliques (16)	0.722 / 0.741 (Borůvka)	0.584 / 0.581 (Borůvka / Kruskal)	0.620 / 0.749 (Kruskal)	1.006 / 1.051 (Kruskal)

The k=256 and k=√n rows at n=65,536 refer to the same generated weighted graphs. This table keeps both observations and does not count them as independent topology evidence.

29. Complete new-case table: spatial and control models

Same aggregation and ratio convention as the preceding page. Spatial random/distance rows share topology, IDs, order and coordinates within each seed. The pairing isolates the weight assignment on this mesh model; it does not turn three graph seeds into a large statistical sample.

Family	4,096	16,384	65,536	262,144
2D grid, random	1.552 / 1.570 (Kruskal)	1.425 / 1.420 (Kruskal)	1.329 / 1.322 (Kruskal)	1.320 / 1.340 (Kruskal)
2D grid, distance	1.721 / 1.737 (Kruskal)	1.564 / 1.519 (Kruskal)	1.451 / 1.445 (Kruskal)	1.459 / 1.505 (Kruskal)
3D grid, random	1.246 / 1.281 (Kruskal)	1.216 / 1.216 (Kruskal)	1.237 / 1.164 (Kruskal)	1.242 / 1.300 (Kruskal)
3D grid, distance	1.382 / 1.373 (Kruskal)	1.305 / 1.318 (Kruskal)	1.262 / 1.257 (Kruskal)	1.334 / 1.325 (Kruskal)
Degree-eight regular	1.308 / 1.311 (Kruskal)	1.336 / 1.346 (Kruskal)	1.436 / 1.396 (Kruskal)	1.712 / 1.815 (Kruskal)
Hub control	0.754 / 0.756 (Kruskal)	0.740 / 0.721 (Kruskal)	0.733 / 0.735 (Kruskal)	0.799 / 0.871 (Kruskal)
Random4 control	1.222 / 1.623 (Kruskal)	1.304 / 1.450 (Kruskal)	1.337 / 1.334 (Kruskal)	1.647 / 1.681 (Kruskal)

For every row, primary analysis and repeat analysis retain KKT/Kruskal, KKT/Prim and KKT/Borůvka ratios, all four runtime summaries, graph-level ranges and individual losses. The detailed CSVs are the numerical source of this report, not values read back from figures.

30. Source and evidence map

Evidence	Location / role
New raw campaigns, work, plan and metadata	common-2026-10-11
New Google Benchmark JSON and every fitted model	complexity analysis
New continuation-strategy experiment	ablation analysis
Exact new graph generators and structural checks	fixtures, tests
Original structural repair campaign	repair-2026-10-09
Six accepted practical optimizations and rejected experiment	practical-2026-10-10
Earlier six additional shapes and controls	shapes-2026-10-11
Standalone offline visual inspection	graph explorer, checked raw exports

Primary algorithm and tooling references:

- Karger, Klein and Tarjan, *A Randomized Linear-Time Algorithm to Find Minimum Spanning Trees*, JACM 42(2), 1995. Author-hosted paper.
- Komlós, *Linear Verification for Spanning Trees*, Combinatorica 5, 1985. Paper text.
- Hagerup, *An Even Simpler Linear-Time Algorithm for Verifying Minimum Spanning Trees*, WG 2009. Publisher record. The audit read the abstract, not a newly obtained full implementation proof.
- Google Benchmark complexity API guide and pinned v1.5.2 fitting code.

The Markdown is the editable report. Static figures show exact retained data or explicitly identified full small instances; the HTML adds local interactions. Neither presentation changes the measured graph inputs, solver code, or correctness requirements.

31. Complete continuation-strategy comparisons

Every ablation case is retained below. Ratios are KKT / repeated production contraction, primary and repeat; below 1 favors KKT. Range columns span the three graph-level ratios, not confidence intervals. The smallest-size observations matter even when the main figure emphasizes the largest tested case.

Family	n	Primary ratio [graph range]	Repeat ratio [graph range]
Bipartite k=16	4,096	1.027 [1.009, 1.043]	1.012 [0.999, 1.012]
Bipartite k=16	65,536	0.989 [0.986, 0.990]	0.991 [0.981, 1.001]
Bipartite k=16	262,144	1.017 [0.960, 1.048]	0.988 [0.975, 1.007]
90% attached leaves	4,096	1.219 [1.197, 1.271]	1.218 [1.214, 1.271]
90% attached leaves	65,536	1.179 [1.176, 1.189]	1.165 [1.160, 1.191]
90% attached leaves	262,144	1.207 [1.205, 1.221]	1.196 [1.194, 1.212]
Ring of cliques (16)	4,096	1.287 [1.234, 1.294]	1.286 [1.279, 1.296]
Ring of cliques (16)	65,536	1.138 [1.099, 1.142]	1.153 [1.092, 1.162]
Ring of cliques (16)	262,144	1.042 [1.036, 1.059]	1.064 [1.061, 1.066]
3D grid, distance	4,096	1.495 [1.471, 1.504]	1.574 [1.548, 1.596]
3D grid, distance	65,536	1.318 [1.304, 1.319]	1.298 [1.295, 1.311]
3D grid, distance	262,144	1.289 [1.278, 1.319]	1.260 [1.247, 1.276]
Degree-eight regular	4,096	1.465 [1.434, 1.527]	1.434 [1.387, 1.449]
Degree-eight regular	65,536	1.324 [1.323, 1.336]	1.335 [1.294, 1.349]
Degree-eight regular	262,144	1.239 [1.216, 1.289]	1.174 [1.109, 1.219]
Random4 control	4,096	1.439 [1.414, 1.450]	1.451 [1.360, 1.495]
Random4 control	65,536	1.325 [1.312, 1.331]	1.251 [1.226, 1.379]
Random4 control	262,144	1.214 [1.191, 1.277]	1.207 [1.167, 1.229]
Small-world variant	4,096	1.394 [1.366, 1.426]	1.385 [1.346, 1.411]
Small-world variant	65,536	1.232 [1.229, 1.239]	1.208 [1.201, 1.233]

Both solvers return the same ID-set representation and use the same production contraction. Compacting Borůvka is deterministic; its sampling-seed labels identify repeated matched trials, not distinct randomized executions.